

Cloud programming

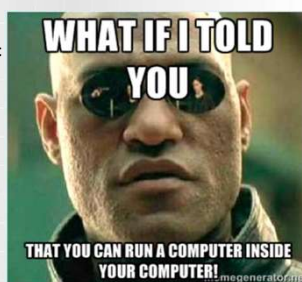
Rafał Palak

Wrocław University of Science and Technology

1

Docker

- A software platform used for automating the deployment process, scaling, and management of applications through containerization. It allows developers to "package" applications along with their dependencies into **standard software units**, called containers, which are **isolated** from the environment and operate uniformly across different systems and infrastructures.



2

Docker - innovativeness

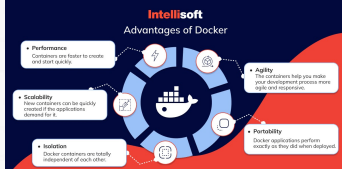
- Portability** - Applications packaged in Docker containers can run consistently across different environments, from developers' laptops to production servers, regardless of the platform.
- Speed and Flexibility** - Containers launch quickly and efficiently, which facilitates the scaling of applications and experimenting with new versions and configurations.
- Application Isolation** - Containerization provides isolation of applications, which reduces dependency conflicts and facilitates version management.
- Resource Optimization** - Thanks to sharing the operating system kernel and a lightweight architecture, Docker enables better resource utilization compared to virtual machines.



3

Docker – key benefits

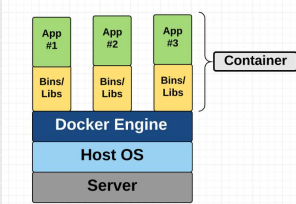
- Enhanced efficiency in development and deployment** - Docker simplifies and accelerates the developer workflow from creation, through testing, to application deployment.
- Ease of management** - Managing containerized applications is simpler due to the use of Docker tools and the ecosystem of solutions supporting containerization.
- Environmental consistency** - Docker ensures that an application will run the same way regardless of where it is launched.



4

Container [1]

- Is a sandboxed process running on a host machine that is isolated from all other processes running on that host machine.

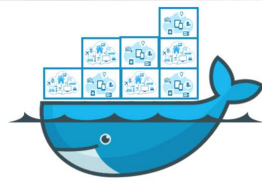


<https://docs.docker.com/get-started/>

5

Container [2]

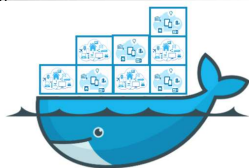
- Isolate application processes** at the host operating system level,
- Use **Linux kernel features** such as **cgroups** and **namespaces**,
- Application in a container has its **own view of the file system, network, users, etc.**, but **still uses the same operating system kernel** on which the container is running,
- It does **not contain its own operating system kernel** nor does it **emulate hardware**,
- It is an **executable instance of an image**.



6

Container [3]

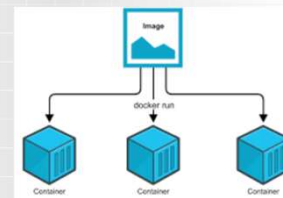
- Because **containers share the operating system** kernel with the host and do **not need to emulate hardware**, they are much **lighter and faster** to launch than virtual machines.
- Containers **start almost instantaneously**, which is a significant advantage in environments where **rapid scaling and flexibility are crucial**.
- Containers can be **described as a form of operating system-level virtualization**, in contrast to traditional virtualization based on virtual machines that emulate an entire hardware environment. As a result, containers can be more efficient in terms of resource usage and performance.



7

Image

- A running container **uses an isolated filesystem**. This isolated filesystem is provided by an image, and the **image must contain everything needed to run an application** - all **dependencies, configurations, scripts, binaries**, etc. The image also contains other configurations for the container, such as environment variables, a default command to run, and other metadata.

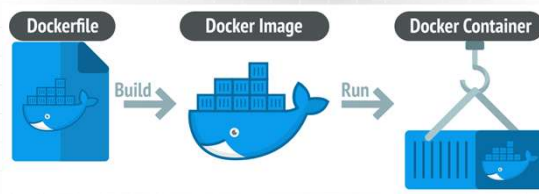


<https://docs.docker.com/get-started/>

8

Image

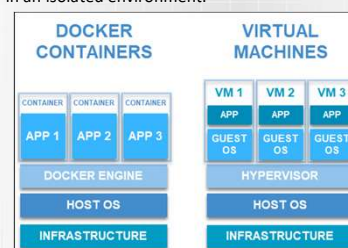
- It defines a mini-system on which we will run a given application.
- Creating such an image is **similar to setting up a server** - we install the necessary packages, configure them, copy files, etc.
- It **consists of several layers**, such as the operating system, e.g., Ubuntu or Alpine, layers related to the application being run, e.g., installed Java/Python, our application.



9

Virtual Machine

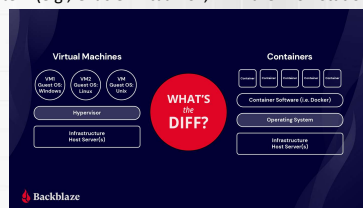
- An **emulated computer with its own operating system**
- Runs on software called a hypervisor**, operating on physical computer hardware
- Allows for multiple operating systems to run on a single physical server**, each in an isolated environment.



10

Virtual Machine - Hypervisor

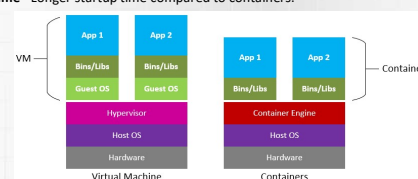
- A program or layer of software that enables hosting one or several VMs, managing the distribution of hardware resources to each virtual machine. There are two types:
 - Type 1 (bare-metal): The software runs directly on physical hardware (e.g., VMware ESXi, Microsoft Hyper-V).
 - Type 2 (hosted): The software runs on the host's operating system (e.g., Oracle VirtualBox, VMware Workstation).



11

Virtual Machine – pros and cons

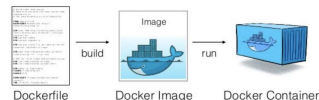
- Isolation** - Virtual machines **offer complete isolation** from the host system and from each other, ensuring security and flexibility.
- Resources** - Sharing of physical host resources; the ability to allocate resources (CPU, RAM, disk space) for each VM.
- Portability** - Ease of migrating virtual machines between physical hosts.
- Redundancy and Fault Tolerance** - Facilitates the creation of high availability environments and continuity planning.
- Resource Consumption** - **Higher resource usage compared to containers**, due to the **need to run a full operating system** for each VM.
- Startup Time** - Longer startup time compared to containers.



12

Dockerfile

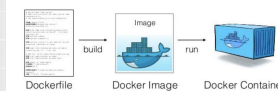
- A plain text file named Dockerfile (no extensions, just the name)
- Contains a series of instructions that allow the creation of a fully functional image that will contain everything needed to run an application.
- Dockerfile consists of pairs (instruction, arguments) for instructions.
- It precisely describes which components should make up the runtime environment for the application placed in the image.



13

Dockerfile – selected instructions

- FROM:** Specifies the base image.
- RUN:** Executes commands in a new layer on top of the current image and commits the results.
- COPY/ADD:** Copies files and directories into the image.
- WORKDIR:** Sets the working directory for the RUN, CMD, ENTRYPOINT, COPY, and ADD instructions.
- CMD:** Sets the default command or options that will be executed when the container is launched.
- EXPOSE:** Informs Docker that the container listens on specified ports at runtime.
- ENV:** Sets environment variables.



14

Dockerfile - example

```

# Instalacja obrazu bazowego na architekturze x86_64, wersja 5.9-jdk17 AS % cache
FROM gcr.io/5.9-jdk17 AS % cache

# Tworzenie katalogu do przechowywania cache'a build
RUN mkdir -p /home/gradle/cache

# Instalacja najnowszej wersji Gradle i ustawienie lokalizacji cache'a build
ENV GRADLE_OPTS="-Xmx2048m -XX:MaxPermSize=256m"
ENV GRADLE_HOME=/home/gradle/gradle
ENV PATH=$PATH:$GRADLE_HOME/bin

# Kopia pliku build.gradle do kontenera build w /usr/src
COPY build.gradle /usr/src

# Instalacja katalogu katalogu roboczego na /usr/src
WORKDIR /usr/src

# Uruchomienie zadania Gradle z celu pobrania zależności i zbudowania projektu, posiadając testy
RUN gradle clean build -x test

# Kopia obrazu Docker z tego samego obrazu bazowego, study do budowy aplikacji
FROM gcr.io/5.9-jdk17 AS % app

# Kopia cache'a build z etapu cache do bieżącego kontenera build
COPY --from=cache /home/gradle/.cache /home/gradle/.cache

# Kopia pliku build.gradle do katalogu roboczego na /usr/src
COPY - /usr/src/build.gradle

# Instalacja katalogu katalogu roboczego na /usr/src
WORKDIR /usr/src

# Budowanie aplikacji Spring Boot, tworzenie JAR-a, przy użyciu cache'a budowania Gradle, posiadając testy
RUN gradle bootJar -x test --build-cache

# Ustawienie zmiennych ENV z domyślną wartością wersji aplikacji
ENV VERSION="0.1.0-SNAPSHOT"

# Ustawienie obrazu bazowego na OpenJDK 17 JRE do uruchomienia aplikacji
FROM eclipse-temurin:17-jre

# Instalowanie katalogu roboczego na /app
WORKDIR /app

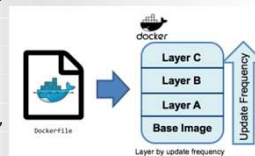
# Kopia pliku JAR do katalogu roboczego
COPY --from=app /usr/src/build-libs.jar /app

# Ustawienie polecenia startowego, które uruchomi aplikację Java z JAR-a
ENTRYPOINT ["java", "-cp", "app.jar", "Main"]
  
```

15

Dockerfile - base image

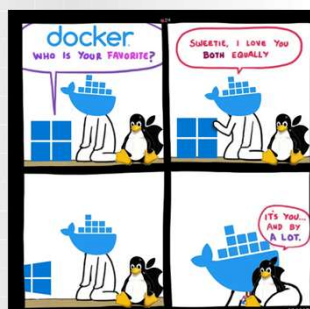
- It acts as a sort of operating system for the image being created.
- It is the base to which we will add further applications (e.g., libraries) and our own code.
- You can use a "bare" operating system, such as Ubuntu or Alpine. In such a case, we must manually install all the tools we need (e.g., Node.js).
- You can use a ready-made image that already has most of these tools pre-installed. As a result, our Dockerfile will be much smaller and easier to manage.



16

.dockerignore

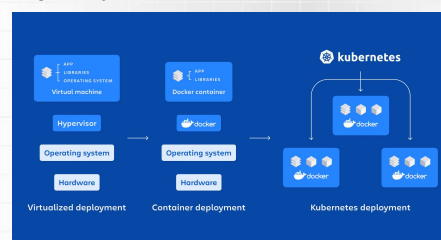
- Specifies which files should not be copied to the image
- Allows for reducing the size of the image
- Works similarly to .gitignore



17

Container management

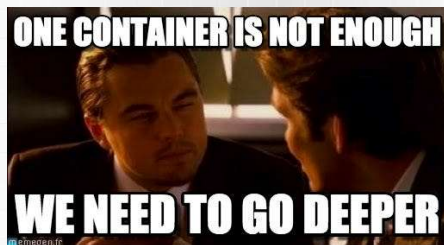
- It is the process of comprehensively managing the life and operation of application containers in an IT environment.
- It encompasses a wide range of activities, from application development through its deployment, scaling, updates, monitoring, to ensuring security.



18

Container Management - Orchestration

- The process of **managing the lifecycle** of multiple containers that together **constitute applications in complex environments**.
- Container orchestration **allows for the automatic deployment, scaling, and state management** of containers.

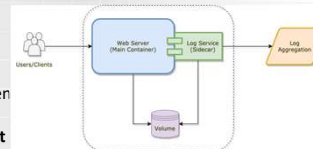


19

19

Container Management - Monitoring and Logging

- Monitoring the state of containers and applications**, as well as **collecting and analyzing logs**, are crucial for maintaining **performance, availability, and security of applications**.
- Container management tools often **integrate with monitoring and logging systems**, providing insight into application operation and facilitating the diagnosis of problems.



20

20

Container Management - Networking

- Network management in a container environment, including **network isolation, load balancing, service discovery, and managing communication between containers**.
- It is **crucial for ensuring effective communication within complex microservices applications**.



21

21

Container Management - Tools

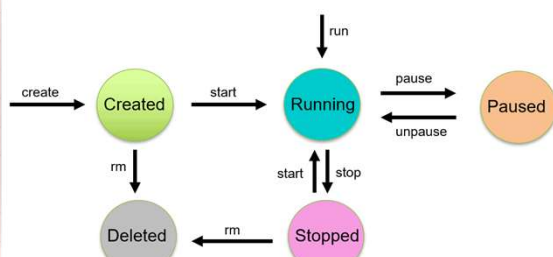
- Docker Swarm** - A tool for clustering and managing Docker containers, providing native Docker cluster management.
- Kubernetes** - An open-source system for automating deployment, scaling, and management of containerized applications.
- OpenShift** - A container application platform from Red Hat, based on Kubernetes, offering additional management and security features.
- Amazon ECS/AKS/GKE** - Container management services provided by major cloud providers: Amazon Elastic Container Service, Azure Kubernetes Service, and Google Kubernetes Engine.



22

22

Docker Image Lifecycle

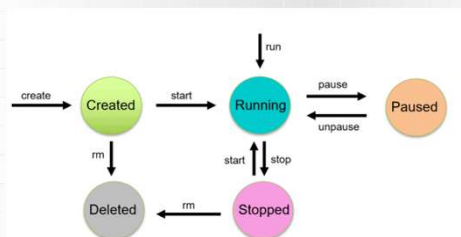


23

23

Basic commands- docker build

- `docker build PATH | URL`
- Builds a Docker image from a Dockerfile

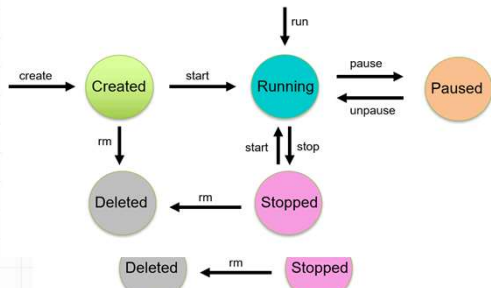


24

24

Basic commands - docker create

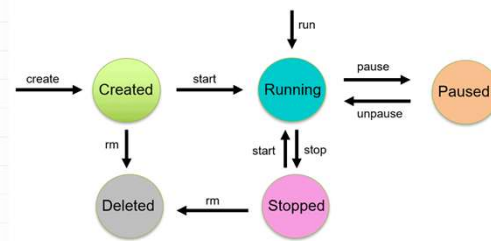
- `docker create --name <container name> <image name>`



25

Basic commands - docker start

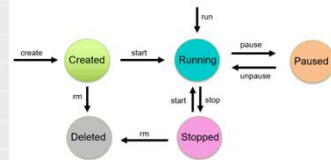
- `docker start <container name>`
- Start a stopped container



26

Basic commands - docker run

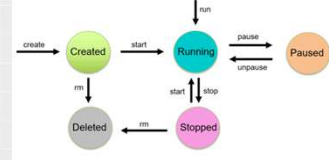
- `docker run <container name>`
- Pulls an image and creates a container from it.
- Executes the action of both "docker create" and "docker start".
- Creates a new container and runs the image in the newly created container.



27

Basic commands - docker pause/unpause

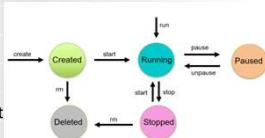
- `docker pause <container name>`
- `docker unpause <container name>`
- Used to temporarily stop all processes inside a specified Docker container.
- When a container is paused, all processes running inside that container are frozen.



28

Basic commands - docker stop

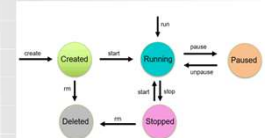
- `docker stop <container name>`
- Stops a running container.
- The process of stopping a container occurs in several stages and is designed to allow for the safe termination of the application and container operations.
- It's the preferred and safe way to stop containers, as it offers applications the opportunity for a clean and controlled shutdown, which is especially important in production environments and for critical applications.
- System resources (e.g., memory, CPU) that were used by the container are released.



29

Basic commands - docker rm

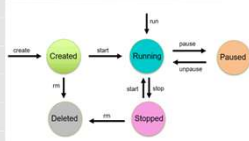
- `docker rm <container name>`
- Used to remove a container from the system.
- Before a container can be deleted, it must be stopped.
- Once the container is stopped, `docker rm` removes its files from the Docker system, including container metadata and any data files associated with the container that are not stored on external volumes, unless the `--volumes` option is used, which also deletes the assigned volumes.
- After the container is deleted, any system resources (i.e., memory, disk space, etc.) are released.



30

Basic commands - docker kill

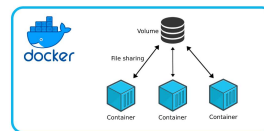
- `docker kill <container name>`
- Used to immediately stop a running container.
- Mainly used in situations where the container does not respond to normal stop commands or when a quick release of system resources occupied by the container is needed.



31

Volumes

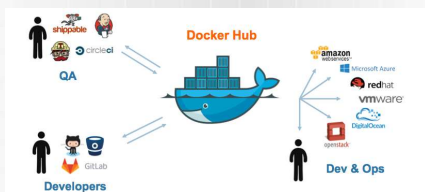
- `docker volume create` – creates a new volume
- `docker volume ls` – allows browsing the list of volumes
- `docker volume inspect` – displays detailed information about a specific volume.
- `docker volume rm` – removes a volume
- `docker run -v my_volume:/app/data my_image`
- This is a mechanism for persistent data storage, used to manage data that should survive the deletion or restart of containers.
- Can be safely shared between multiple containers



32

DockerHub

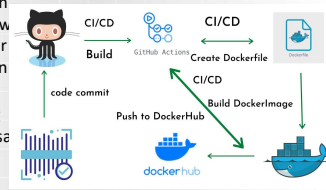
- It's a cloud service that allows **sharing and managing Docker image repositories**. It is the **official image hub for Docker**, containing both public and private repositories.



33

DockerHub – key functions [1]

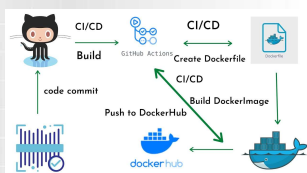
- **Repositories** - Ability to create public and private repositories for storing Docker images.
- **Automatic Building** - Integration with GitHub and Bitbucket, allow for automatic building of Docker images upon any code change in repository.
- **Image Versioning** - Maintaining different versions (tags) of the same image, which facilitates version management and rollbacks.



34

DockerHub – key functions [2]

- **Official Images** - Access to a wide range of official Docker images maintained by software creators and verified by Docker Inc.
- **Community** - Access to images created by the community, which can be used as a base for your own applications.



35

Docker Compose

- Docker Compose is a tool for **defining and running multi-container Docker applications** using YAML files.
- It allows **configuring services, networks, and volumes** in a single file, **simplifying the deployment** process of complex applications.
- It **enables the automatic creation of networks between containers**, **facilitating communication** between services without the need for manual configuration.

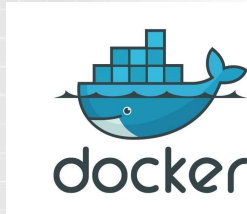
```

1 version: '3.3'
2 services:
3   db:
4     image: mysql:5.7
5     volumes:
6       - db_data:/var/lib/mysql
7     restart: always
8     environment:
9       MYSQL_ROOT_PASSWORD: rootpassword
10    MYSQL_DATABASE: wordpress
11    MYSQL_USER: wordpress
12    MYSQL_PASSWORD: wordpress
13  wordpress:
14    depends_on: db
15    image: wordpress:latest
16    ports:
17      - "8080:80"
18    restart: always
19  environment:
20    WORDPRESS_DB_HOST: db:3306
21    WORDPRESS_DB_USER: wordpress
22    WORDPRESS_DB_PASSWORD: wordpress
23  volumes:
24    db_data:
  
```

36

Optimization of Docker images

- **Lightweight Base Images** - Use lightweight base images, such as **Alpine Linux**, which are smaller in size and contain fewer unnecessary files and libraries.
- **Official Images** - Prefer official images available on Docker Hub, which are **regularly updated and optimized for security and performance**.
- **Minimization of Layer Count** - Combine RUN commands into a single chain using logical operators (&&) to reduce the number of layers.

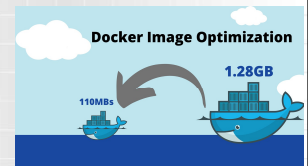


37

37

Optimization of Docker images

- **Removing Unnecessary Files** - Clean unnecessary files and packages in the same RUN command that installs them to avoid saving them in image layers.
- **Ignoring Unnecessary Files** - Add a `.dockerignore` file to your project to exclude unnecessary files and directories in the image (e.g., development dependencies, temporary files).



38

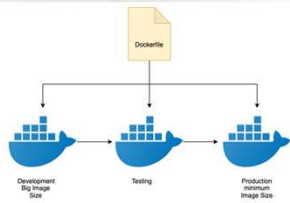
38

Optimization of Docker images

- **Build Optimization** - Use multi-stage builds to separate building and runtime environments. This allows including only the final artifacts in the target image, reducing its size.
- **Dependency Minimization** - Install only necessary packages and tools to decrease the image size and limit the attack surface.

```

1 # Stage - Development
2 FROM dev-image as dev
3 ...
4 ... stage 0
5
6 # Stage - Testing
7 FROM Test-Image as test
8 COPY --from=dev /src/app .
9 ...
10 ... stage 1
11
12 # Stage - Production
13 FROM Minimum-Image as prod
14 COPY --from=dev /src/app .
15 ...
16 ... stage 2
  
```

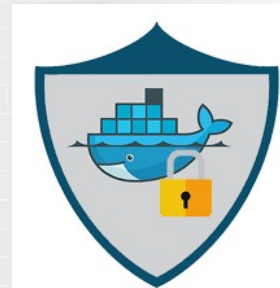


39

39

Container Security

- Storing sensitive data in environment variables
- `docker run -e "PASSWORD=myPassword" application`



40

40

Thank you for your
attention

41

41